SkillNavora

STAGE 1 HANDBOOK

Stage 1: Programming foundations

Outcome: Understand programming; build simple applications.
Topics: Computer and software basics, Python, SQL, Git, APIs, JavaScript basics, Terminal, Debugging.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.
Financial figures are hypothetical examples, not advice.

Stage 1: Programming foundations

Outcome: Understand programming; build simple applications

This handbook covers each topic of Stage 1 with a definition, why it matters, how it works, step-by-step instructions, example code or templates, common mistakes, interview questions with answers, a hands-on exercise and official resources.

Topics in this stage

1. Computer and software basics
2. Python
3. SQL
4. Git
5. APIs
6. JavaScript basics
7. Terminal
8. Debugging

STAGE 1 · TOPIC 1

Computer and software basics

DEFINITION

> A computer is a machine that stores data and follows instructions. Hardware is the physical part (CPU, memory, storage, network card). Software is the instructions: the operating system (Windows, macOS, Linux) manages the hardware, and applications (browser, Excel, your Python app) do useful work on top of it.

WHY IT MATTERS

> Every AI system you deploy runs on computers: laptops, servers, containers in the cloud. Knowing what CPU, memory, disk, processes, files and networks are lets you understand errors such as 'out of memory', 'port already in use', 'permission denied' or 'connection refused' instead of guessing.

HOW IT WORKS

- CPU: executes instructions. More cores run more work in parallel. GPUs run thousands of small calculations at once, which is why AI training and local model inference use them.
- Memory (RAM): fast, temporary space for running programs. When a program needs more than is free, it slows down or crashes.
- Storage (SSD/HDD): keeps files permanently. Files live in folders; a path such as /home/asha/project/app.py or C:\Users\Asha\project\app.py tells the system where.
- Operating system: starts programs as processes, gives each its memory, controls file permissions and drivers, and manages users.
- Network: computers find each other by IP address (or a domain name resolved by DNS) and talk on ports (443 for HTTPS, 5432 for PostgreSQL, 8000 often for a local API). Data travels in requests and responses under protocols such as HTTP.

- Client and server: a client asks (your browser); a server answers (a web server in the cloud). The same machine can be both.

STEP-BY-STEP

1. Open your system monitor (Task Manager on Windows, Activity Monitor on macOS, top/htop on Linux) and find CPU, memory and running processes.
2. Open a terminal and print your current folder (pwd on macOS/Linux, cd on Windows). List files (ls or dir).
3. Look up your IP address and ping a website (ping google.com) to see network round-trip time.
4. Visit https://example.com, open browser DevTools (F12) → Network tab, reload, and read the request URL, method (GET), status (200) and response headers.
5. Write down, in your own words, what happens between typing a URL and seeing the page (DNS → TCP/TLS connection → HTTP request → server response → browser renders).

EXAMPLE CODE / TEMPLATE

```
# What happens when your app calls an AI API (conceptual)
# 1. Your code (a process) builds a JSON request in memory
# 2. DNS turns api.example.com into an IP address
# 3. A TLS-encrypted connection opens to port 443
# 4. An HTTP POST carries the JSON body
# 5. The server runs the model and returns JSON
# 6. Your code parses the reply and shows it
```

COMMON MISTAKES

- Confusing memory (RAM) with storage (disk).
- Ignoring error messages; they almost always name the resource (file, port, permission) that failed.
- Running everything as administrator/root, which hides permission problems and is unsafe.

INTERVIEW QUESTIONS AND ANSWERS

Q. What is the difference between a process and a program?

A. A program is the code stored on disk; a process is a running instance of it in memory with its own ID, memory and resources. One program can run as several processes.

Q. What happens when you type a URL into a browser?

A. The browser resolves the domain with DNS to an IP, opens a TCP connection (with TLS for HTTPS), sends an HTTP request, the server returns a response (HTML, JSON), and the browser renders it, fetching further resources as needed.

Q. Why do AI workloads use GPUs?

A. Neural networks are mostly large matrix multiplications that can be done in parallel; GPUs have thousands of cores designed for exactly that, so they are far faster than CPUs for training and inference.

HANDS-ON EXERCISE

> Draw a diagram of your laptop calling an AI API: label the process, memory, DNS, port 443, HTTPS request, server and JSON response.

OFFICIAL AND RECOMMENDED RESOURCES

- MDN: How the web works https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_Web_works
- CS50x (free Harvard intro to computer science) https://cs50.harvard.edu/x/

STAGE 1 · TOPIC 2

Python

DEFINITION

Python is a general-purpose programming language with simple, readable syntax. It is the main language for AI, data and automation because of its libraries: requests (HTTP), pandas (data), FastAPI (web APIs), and the official SDKs for OpenAI, Anthropic and Google.

WHY IT MATTERS

Most FDE work is glue code: read data, call an API or model, transform the result, save it, and expose it to users. Python lets you do all of that quickly, and almost every AI example, framework and SDK is published for Python first.

HOW IT WORKS

- Python runs your .py file top to bottom through the interpreter (python app.py).
- Values have types: int, float, str, bool, None, list, tuple, set, dict. Python works out the type at run time.
- Indentation (4 spaces) defines code blocks for if, for, while, def and class.
- Functions package logic; modules (files) and packages (folders) organise it; pip installs third-party packages into a virtual environment.
- Errors raise exceptions; try/except catches them so you can log, retry or skip a bad record.

STEP-BY-STEP

1. Install Python 3.12+ from python.org and VS Code with the Python extension.
2. Create a folder lead-tool, open it in VS Code, and run python -m venv .venv then activate it (.venv\Scripts\activate on Windows, source .venv/bin/activate on macOS/Linux).
3. Create leads.csv with columns name,email,budget and a few rows (leave one email empty).
4. Write the script below, run python clean_leads.py, and check clean_leads.json.
5. Add a function score(budget) that returns 'hot' if budget ≥ 5000000 else 'warm', and include it in the output.
6. Save dependencies: pip freeze > requirements.txt.

EXAMPLE CODE / TEMPLATE

```
import csv, json
from pathlib import Path

def load_leads(path: str) → list[dict]:
    with open(path, newline='', encoding='utf-8') as f:
        return list(csv.DictReader(f))

def is_valid(lead: dict) → bool:
    return bool(lead.get('email')) and '@' in lead['email']

def score(budget: str) → str:
    try:
        return 'hot' if int(budget) ⩾ 5_000_000 else 'warm'
    except ValueError:
        return 'unknown'

leads = load_leads('leads.csv')
clean = [{**l, 'score': score(l['budget'])} for l in leads if is_valid(l)]
```

```
Path('clean_leads.json').write_text(json.dumps(clean, indent=2))
print(f'{len(clean)} of {len(leads)} leads kept')
```

COMMON MISTAKES

- Installing packages globally instead of in a virtual environment.
- Mixing tabs and spaces, causing IndentationError.
- Catching every exception with a bare except: and hiding real bugs.
- Putting API keys directly in code; use environment variables (os.environ['API_KEY']).

INTERVIEW QUESTIONS AND ANSWERS

Q. What is the difference between a list and a tuple?

A. Both are ordered sequences. A list is mutable (you can add, remove, change items); a tuple is immutable. Use tuples for fixed records and as dictionary keys.

Q. What does a virtual environment solve?

A. It isolates each project's installed packages and versions so projects do not conflict, and lets you reproduce the environment from requirements.txt.

Q. How do you handle a file that may not exist?

A. Check with pathlib.Path(p).exists() or wrap open() in try/except FileNotFoundError and handle it (log, create, or skip).

Q. What is a list comprehension?

A. A compact way to build a list from an iterable with optional filtering, e.g. [x*2 for x in nums if x > 0].

HANDS-ON EXERCISE

> Extend the script to read an Excel file with pandas, remove duplicate emails, and write a summary count of hot, warm and unknown leads.

OFFICIAL AND RECOMMENDED RESOURCES

- Python tutorial (official) https://docs.python.org/3/tutorial/
- Python venv (official) https://docs.python.org/3/library/venv.html
- Automate the Boring Stuff (free book) https://automatetheboringstuff.com/

SQL

DEFINITION

SQL (Structured Query Language) reads and changes data in relational databases such as PostgreSQL, MySQL and SQLite. Data lives in tables of rows and columns, linked by keys.

WHY IT MATTERS

Enterprise data almost always sits in SQL databases. FDEs write queries to understand a customer's data, build features, feed AI with context, check results and create reports. SQL is tested in almost every FDE and data interview.

HOW IT WORKS

- CREATE TABLE defines columns and types; PRIMARY KEY makes rows unique; FOREIGN KEY links tables.
- INSERT adds rows; UPDATE changes them; DELETE removes them (always with a WHERE clause).
- SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT is the shape of a read query.
- JOINs combine tables: INNER JOIN keeps matching rows; LEFT JOIN keeps all rows from the left table.
- CTEs (WITH) name steps; window functions (OVER (PARTITION BY … ORDER BY …)) compute ranks and running totals without collapsing rows.
- Indexes make lookups fast; transactions make multi-step changes all-or-nothing.

STEP-BY-STEP

1. Install PostgreSQL or run it with Docker: docker run --name pg -e POSTGRES_PASSWORD=secret -p 5432:5432 -d postgres.
2. Connect with psql or DBeaver and create the tables below.
3. Insert 10 sample customers and 30 orders.
4. Write queries: total spend per customer, top 3 customers per city, customers with no orders.
5. Run EXPLAIN on a slow query, add an index, and compare.

EXAMPLE CODE / TEMPLATE

```
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  city TEXT
);
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  customer_id INT REFERENCES customers(id),
  amount NUMERIC(10,2),
  created_at DATE DEFAULT CURRENT_DATE
);

-- Total spend per customer, highest first
SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON o.customer_id = c.id
GROUP BY c.name ORDER BY total DESC;

-- Top 3 customers per city (window function)
```

```
WITH spend AS (
  SELECT c.city, c.name, SUM(o.amount) AS total
  FROM customers c JOIN orders o ON o.customer_id = c.id
  GROUP BY c.city, c.name
)
SELECT * FROM (
  SELECT *, RANK() OVER (PARTITION BY city ORDER BY total DESC) AS rnk FROM spend
) t WHERE rnk ⩽ 3;

-- Customers with no orders
SELECT c.* FROM customers c LEFT JOIN orders o ON o.customer_id = c.id WHERE o.id IS NULL;
```

COMMON MISTAKES

- UPDATE or DELETE without WHERE, changing every row.
- Using WHERE to filter aggregates (use HAVING).
- Building SQL by joining strings with user input, which allows SQL injection; use parameters.
- Selecting * in production queries that only need a few columns.

INTERVIEW QUESTIONS AND ANSWERS

Q. Difference between WHERE and HAVING?

A. WHERE filters rows before grouping; HAVING filters groups after aggregation, e.g. HAVING SUM(amount) > 10000.

Q. INNER JOIN vs LEFT JOIN?

A. INNER JOIN returns only rows with a match in both tables; LEFT JOIN returns every row from the left table and NULLs where the right has no match.

Q. What is a window function?

A. A function computed over a set of related rows (the window) while keeping each row, e.g. RANK() OVER (PARTITION BY city ORDER BY total DESC).

Q. How do you prevent SQL injection?

A. Use parameterised queries or an ORM so user input is sent as data, never concatenated into SQL text; also validate input and use least-privilege database users.

HANDS-ON EXERCISE

> Design a schema for a real-estate CRM (agents, leads, properties, visits) and write five business queries such as conversion rate per agent.

OFFICIAL AND RECOMMENDED RESOURCES

- PostgreSQL tutorial (official) https://www.postgresql.org/docs/current/tutorial.html
- PostgreSQL window functions (official) https://www.postgresql.org/docs/18/tutorial-window.html
- SQLBolt (interactive lessons) https://sqlbolt.com/
- LeetCode SQL 50 https://leetcode.com/studyplan/top-sql-50/

Git

DEFINITION

Git is a version control system that records every change to your project as commits, so you can see history, work on branches, collaborate and undo mistakes. GitHub hosts Git repositories online with pull requests, reviews, issues and CI.

WHY IT MATTERS

Every professional team uses Git. Customers will review your code through pull requests, CI runs from GitHub, and your portfolio lives on GitHub. Without Git you cannot collaborate or recover safely from mistakes.

HOW IT WORKS

- Working directory (your files) → staging area (git add) → repository history (git commit).
- A branch is a movable pointer to a line of commits; main is the default branch.
- A remote (origin) is the copy on GitHub; push uploads commits and pull downloads and merges them.
- A pull request proposes merging a branch, with review and automated checks.
- .gitignore lists files Git must not track, such as .env, .venv and node_modules.

STEP-BY-STEP

1. Install Git and set your name and email: git config --global user.name 'Your Name'.
2. In your project: git init, create .gitignore (.venv/, .env, __pycache__/), git add ., git commit -m 'Initial commit'.
3. Create an empty repository on GitHub and push: git remote add origin URL; git push -u origin main.
4. Create a branch: git switch -c feature/scoring; edit code; commit; git push -u origin feature/scoring.
5. Open a pull request on GitHub, review the diff, merge it, then git switch main && git pull.

EXAMPLE CODE / TEMPLATE

```
git init
echo -e '.venv/\n.env\n__pycache__/' > .gitignore
git add .
git commit -m 'Initial commit: lead cleaner'
git branch -M main
git remote add origin https://github.com/<you>/lead-tool.git
git push -u origin main

git switch -c feature/scoring
# ...edit files...
git add clean_leads.py
git commit -m 'Add lead scoring by budget'
git push -u origin feature/scoring
# open a pull request on GitHub, then after merge:
git switch main && git pull
```

COMMON MISTAKES

- Committing secrets (.env, API keys). If it happens, revoke and rotate the key; removing it from history is not enough.
- Huge commits with vague messages ('fix').

- Working directly on main in a team.
- Force-pushing over teammates' work.

INTERVIEW QUESTIONS AND ANSWERS

Q. What is the difference between git merge and git rebase?
A. Merge joins two branches with a merge commit and keeps history as it happened; rebase replays your commits on top of another branch for a linear history. Do not rebase commits others already use.

Q. How do you resolve a merge conflict?
A. Open the conflicted files, choose or combine the changes between the conflict markers, remove the markers, test, then git add and commit.

Q. What should never be committed?
A. Secrets and credentials, local environment folders, build outputs and large data files; list them in .gitignore.

HANDS-ON EXERCISE

> Publish your lead tool on GitHub with a README (problem, setup, usage, screenshot) and at least three meaningful commits from separate branches.

OFFICIAL AND RECOMMENDED RESOURCES

- Pro Git book (official, free) https://git-scm.com/book/en/v2
- GitHub Docs: get started https://docs.github.com/en/get-started
- Learn Git Branching (interactive) https://learngitbranching.js.org/

APIs

DEFINITION

An API (application programming interface) is a defined way for programs to talk. Web APIs accept HTTP requests at URLs (endpoints) and return data, usually JSON. AI models, CRMs, payment systems and messaging platforms are all used through APIs.

WHY IT MATTERS

An FDE's main job is connecting systems: the customer's CRM, documents and databases to AI models and back. Reading API docs, authenticating, sending requests, handling errors and limits is daily work.

HOW IT WORKS

- Endpoint: a URL such as https://api.example.com/v1/leads.
- Method: GET (read), POST (create), PUT/PATCH (update), DELETE (remove).
- Headers: metadata such as Content-Type: application/json and Authorization: Bearer <token>.
- Body: JSON data for POST/PUT/PATCH.
- Status codes: 200 OK, 201 Created, 400 bad request, 401 not authenticated, 403 forbidden, 404 not found, 422 validation error, 429 too many requests, 500 server error.
- Webhooks reverse the direction: the other system calls your URL when an event happens.

STEP-BY-STEP

1. Read the API's documentation: base URL, authentication, endpoints, limits.
2. Test a call in Postman or with curl to see the real response.
3. Store the key in an environment variable or .env file (never in code).
4. Write a small Python client with timeouts and error handling.
5. Handle pagination and 429 rate limits with retries and backoff.
6. Log failures with status code and request ID, without logging secrets.

EXAMPLE CODE / TEMPLATE

```
import os, time, requests

BASE = 'https://jsonplaceholder.typicode.com'  # free practice API

def get_json(path: str, retries: int = 3) → dict | list:
    for attempt in range(retries):
        r = requests.get(f'{BASE}{path}', timeout=10,
                         headers={'Authorization': f"Bearer {os.environ.get('API_TOKEN', '')}"})
        if r.status_code == 429:           # rate limited: wait and retry
            time.sleep(2 ** attempt)
            continue
        r.raise_for_status()               # raises for 4xx/5xx
        return r.json()
    raise RuntimeError('Too many retries')

posts = get_json('/posts?userId=1')
print(len(posts), 'posts; first title:', posts[0]['title'])

# Create (POST) a record
```

```
r = requests.post(f'{BASE}/posts', json={'title': 'Hello', 'body': 'From FDE', 'userId': 1},
timeout=10)
print(r.status_code, r.json())
```

COMMON MISTAKES

- No timeout, so a slow API freezes your app.
- Retrying non-idempotent POSTs blindly and creating duplicates.
- Ignoring pagination and only reading the first page.
- Logging full request headers including tokens.

INTERVIEW QUESTIONS AND ANSWERS

Q. REST vs GraphQL?

A. REST exposes resources at many URLs with fixed response shapes; GraphQL exposes one endpoint where clients query exactly the fields they need. REST is simpler and cache-friendly; GraphQL reduces over- and under-fetching.

Q. What does HTTP 429 mean and how do you handle it?

A. Too many requests: you hit a rate limit. Wait (respect the Retry-After header if present) and retry with exponential backoff; reduce concurrency or batch requests.

Q. What is idempotency?

A. An operation that has the same effect if repeated. GET, PUT and DELETE should be idempotent; for POST, use idempotency keys so retries do not create duplicates.

Q. How do you secure a webhook endpoint?

A. Use HTTPS, verify the sender's signature with the shared secret, check timestamps to block replays, and respond quickly while processing in the background.

HANDS-ON EXERCISE

> Build a script that reads a public API page by page, stores the results in SQLite, and re-runs safely without duplicates.

OFFICIAL AND RECOMMENDED RESOURCES

- MDN: HTTP overview https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- Requests library docs https://requests.readthedocs.io/en/latest/
- Postman learning center https://learning.postman.com/docs/introduction/overview/

JavaScript basics

DEFINITION

> JavaScript is the programming language of web browsers and, through Node.js, of many servers. It makes pages interactive and is the base of React, Next.js and TypeScript.

WHY IT MATTERS

> Most AI products have a web front end. Even backend-focused FDEs read and adjust JavaScript/TypeScript to fix a UI, add a chat widget, call an API from the browser or debug what a customer sees.

HOW IT WORKS

- Variables: const (cannot be reassigned) and let. Types: string, number, boolean, null, undefined, object, array.
- Functions: function f(x) {} or arrow functions const f = (x) ⇒ x * 2.
- Arrays and objects: [1,2,3].map(), .filter(), .find(); {name: 'Asha'} with dot access.
- The DOM is the page as objects; document.querySelector finds elements and addEventListener reacts to clicks.
- Async: fetch returns a Promise; await waits for it inside an async function.
- Modules: export and import share code between files.

STEP-BY-STEP

1. Install Node.js LTS and check node -v and npm -v.
2. Create index.html with an input, a button and a list.
3. Add a script that fetches posts from a practice API and shows the titles.
4. Open DevTools Console to see errors and console.log output.
5. Move the logic into app.js and load it with <script type="module" src="app.js">.

EXAMPLE CODE / TEMPLATE

```
<!-- index.html -->
<input id="q" placeholder="Filter titles">
<button id="go">Load</button>
<ul id="list"></ul>
<script type="module">
  const list = document.querySelector('#list');
  document.querySelector('#go').addEventListener('click', async () => {
    list.textContent = 'Loading…';
    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/posts');
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const posts = await res.json();
      const q = document.querySelector('#q').value.toLowerCase();
      list.innerHTML = '';
      posts.filter(p => p.title.includes(q)).slice(0, 10).forEach(p => {
        const li = document.createElement('li');
        li.textContent = p.title;   // textContent avoids HTML injection
        list.append(li);
      });
    } catch (err) {
```

```
      list.textContent = 'Could not load: ' + err.message;
    }
  });
</script>
```

COMMON MISTAKES

- Using == instead of === (loose comparison causes surprises).
- Forgetting await, so you get a Promise instead of data.
- Inserting user text with innerHTML, which allows script injection (XSS).
- Putting secret API keys in browser code, where anyone can read them.

INTERVIEW QUESTIONS AND ANSWERS

Q. let vs const vs var?

A. const cannot be reassigned, let can; both are block-scoped. var is function-scoped and hoisted, so modern code avoids it.

Q. What is a Promise?

A. An object representing a value that will be available later (or an error). You consume it with .then/.catch or await inside an async function.

Q. Why must API keys not be in frontend code?

A. Everything shipped to the browser can be read by users; keys must stay on the server, which calls the API on the user's behalf.

HANDS-ON EXERCISE

> Build a small page that calls your own backend endpoint and shows a loading state, results and an error message.

OFFICIAL AND RECOMMENDED RESOURCES

- MDN JavaScript guide https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
- javascript.info (free) https://javascript.info/
- Node.js learn https://nodejs.org/en/learn

Terminal

DEFINITION

> The terminal (command line, shell) is a text interface for running programs and managing files: bash or zsh on macOS/Linux, PowerShell on Windows. Servers, containers and cloud tools are mostly operated from it.

WHY IT MATTERS

> Installing packages, running servers, using Git, Docker, kubectl and cloud CLIs, reading logs on a server and automating tasks all happen in the terminal. Customers' production servers rarely have a graphical interface.

HOW IT WORKS

- The prompt waits for a command; a command has a name, options (flags) and arguments: ls -la /var/log.
- Navigation: pwd (where am I), ls (list), cd (change folder), mkdir, rm, cp, mv.
- Reading files: cat, less, head, tail -f (follow a log), grep (search text).
- Environment variables: export API_KEY=... (macOS/Linux) or $env:API_KEY='...' (PowerShell).
- Pipes (|) send one command's output into another; > writes output to a file.
- Processes: ps, top, kill; Ctrl+C stops the running command.

STEP-BY-STEP

1. Open a terminal and run pwd, ls, mkdir demo, cd demo.
2. Create a file: echo 'hello' > note.txt; read it with cat note.txt.
3. Search: grep -i error app.log | tail -n 20.
4. Set an environment variable and read it in Python with os.environ.
5. Run a local server (python -m http.server 8000) and open http://localhost:8000; stop it with Ctrl+C.
6. Find what is using a port: lsof -i :8000 (macOS/Linux) or netstat -ano | findstr 8000 (Windows).

EXAMPLE CODE / TEMPLATE

```
# macOS / Linux (bash or zsh)
pwd                       # print working directory
ls -la                    # list all files with details
mkdir -p projects/fde && cd projects/fde
python -m venv .venv && source .venv/bin/activate
pip install requests
export API_TOKEN='paste-token-here'   # set for this session only
grep -n 'ERROR' app.log | tail -n 20 # last 20 error lines
tail -f app.log           # follow a log live (Ctrl+C to stop)
curl -s https://api.github.com | head -n 5

# Windows PowerShell equivalents
# Get-Location; Get-ChildItem; Set-Location projects\fde
# python -m venv .venv; .venv\Scripts\Activate.ps1
# $env:API_TOKEN = 'paste-token-here'
# Select-String -Path app.log -Pattern 'ERROR' | Select-Object -Last 20
```

COMMON MISTAKES

- Running rm -rf or deleting without checking the path.
- Pasting commands from the internet without understanding them.
- Saving secrets in shell history; prefer .env files loaded by the app.
- Forgetting to activate the virtual environment.

INTERVIEW QUESTIONS AND ANSWERS

Q. How do you find and stop a process using port 8000?
A. Find its PID with lsof -i :8000 (or netstat on Windows), then stop it with kill <PID> (or Stop-Process).

Q. What does a pipe do?
A. It sends the output of one command as input to the next, e.g. cat app.log | grep ERROR | wc -l counts error lines.

Q. How do you follow a log file as it grows?
A. tail -f filename (Get-Content -Wait on PowerShell).

HANDS-ON EXERCISE

> Write a shell script setup.sh that creates a virtual environment, installs requirements and starts your API; document it in the README.

OFFICIAL AND RECOMMENDED RESOURCES

- The Missing Semester (MIT, free) https://missing.csail.mit.edu/
- Microsoft: PowerShell docs https://learn.microsoft.com/powershell/scripting/overview
- explainshell (explains commands) https://explainshell.com/

Debugging

DEFINITION

> Debugging is the systematic process of finding why software behaves wrongly and fixing the cause. It uses error messages, logs, breakpoints, tests and experiments.

WHY IT MATTERS

> In customer deployments things break in ways that never happened on your laptop: different data, permissions, networks and versions. A calm, systematic debugging method is what makes an FDE trusted.

HOW IT WORKS

- Reproduce: get the bug to happen reliably with the smallest input.
- Read the error: the last line of a Python traceback names the exception; the lines above show where it happened.
- Form a hypothesis and test one change at a time.
- Inspect state: print/log variables or set breakpoints in a debugger and step through.
- Isolate: remove parts until the bug disappears; binary-search through recent changes (git bisect).
- Fix the cause, add a test that would have caught it, and confirm the fix.

STEP-BY-STEP

1. Run the failing script and copy the full traceback.
2. Find the top line inside your own code and open that file and line.
3. In VS Code, click left of the line number to set a breakpoint and press F5 (Run and Debug).
4. Step over (F10) and into (F11); watch variables in the left panel.
5. When the bad value appears, ask why it was produced; fix that.
6. Write a pytest test for the case and run pytest.

EXAMPLE CODE / TEMPLATE

```
import logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')
log = logging.getLogger('leads')

def average_budget(leads: list[dict]) → float:
    budgets = []
    for lead in leads:
        try:
            budgets.append(int(lead['budget']))
        except (KeyError, ValueError) as e:
            log.warning('Skipping lead %s: bad budget %r (%s)', lead.get('name'), lead.get('budget'),
e)
    if not budgets:                 # bug fixed: division by zero on empty input
        return 0.0
    return sum(budgets) / len(budgets)

# test_leads.py  (run: pytest)
def test_average_ignores_bad_rows():
    assert average_budget([{'name': 'A', 'budget': '100'}, {'name': 'B', 'budget': 'n/a'}]) == 100
```

```
def test_average_empty():
    assert average_budget([]) == 0.0
```

COMMON MISTAKES

- Changing many things at once, so you do not know what fixed it.
- Fixing the symptom (catching and ignoring the error) instead of the cause.
- Not reading the whole error message.
- Debugging in production without logs or a way to reproduce.

INTERVIEW QUESTIONS AND ANSWERS

Q. Walk me through how you debug a production issue.

A. Assess impact and mitigate first (rollback or feature flag), gather evidence (logs, metrics, traces, recent changes), reproduce in a safe environment, form and test hypotheses, fix the root cause, add a test and monitoring, and write a short post-mortem.

Q. What is a stack trace?

A. The list of function calls active when an error occurred, from the entry point to the failing line; it shows where and how the program got there.

Q. How do you debug an LLM feature that gives bad answers?

A. Log the exact prompt, context, model and parameters; reproduce with the same inputs; check whether retrieval returned the right documents; vary one thing (prompt, context, model) at a time; and add the case to the evaluation set.

HANDS-ON EXERCISE

> Take a script with three planted bugs (wrong key, off-by-one, empty input) and find each using the debugger, then add a test for each.

OFFICIAL AND RECOMMENDED RESOURCES

- VS Code: Python debugging https://code.visualstudio.com/docs/python/debugging
- Python logging HOWTO (official) https://docs.python.org/3/howto/logging.html
- pytest documentation https://docs.pytest.org/en/stable/